Step 0 — Mount Drive and Setup

Reused from Week 2 — updated folder structure for per-document indexing.

```
from google.colab import drive
drive.mount('/content/drive')

import os

PROJECT_DIR = '/content/drive/MyDrive/doctalk'
STANDALONE_DIR = f'{PROJECT_DIR}/faiss_index/standalone'
DOCS_DIR = f'{PROJECT_DIR}/uploaded_docs'

os.makedirs(STANDALONE_DIR, exist_ok=True)
os.makedirs(DOCS_DIR, exist_ok=True)

print('Google Drive mounted and project folders ready.')
print(f'Project folder: {PROJECT_DIR}')
```

```
Mounted at /content/drive
Google Drive mounted and project folders ready.
Project folder: /content/drive/MyDrive/doctalk
```

Step 1 — Install Dependencies

Reused from Week 2 — added python-docx and docx2txt for DOCX support.

```
!pip install -q langchain langchain-community langchain-huggingface
!pip install -q langchain-text-splitters
!pip install -q faiss-cpu
!pip install -q pymupdf
!pip install -q sentence-transformers
!pip install -q groq langchain-groq
!pip install -q python-docx docx2txt

print('All dependencies installed.')
```

```
   ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 2.5/2.5 MB 29.6 MB/s eta 0:00:00
   ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 1.0/1.0 MB 41.3 MB/s eta 0:00:00
   ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 64.9/64.9 kB 4.8 MB/s eta 0:00:00
   ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 51.0/51.0 kB 3.5 MB/s eta 0:00:00
ERROR: pip's dependency resolver does not currently take into account all the packages that are installed. This be
google-colab 1.0.0 requires requests==2.32.4, but you have requests 2.33.1 which is incompatible.
   ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 23.8/23.8 MB 69.8 MB/s eta 0:00:00
   ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 25.0/25.0 MB 55.7 MB/s eta 0:00:00
   ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 137.5/137.5 kB 3.9 MB/s eta 0:00:00
   ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 253.0/253.0 kB 5.1 MB/s eta 0:00:00
All dependencies installed.
```

Step 2 — Load Embedding Model and LLM

Reused from Week 2 — no changes.

```
from langchain_huggingface import HuggingFaceEmbeddings
from langchain_groq import ChatGroq
from google.colab import userdata

print('Loading embedding model...')
embeddings = HuggingFaceEmbeddings(
    model_name='all-MiniLM-L6-v2',
    model_kwargs={'device': 'cpu'},
    encode_kwargs={'normalize_embeddings': True}
)
print('Embedding model loaded.')

GROQ_API_KEY = userdata.get('GROQ_API_KEY')
```

```
GROQ_API_KEY = userdata.get('GROQ_API_KEY')
llm = ChatGroq(
    api_key=GROQ_API_KEY,
    model_name='llama-3.3-70b-versatile',
    temperature=0
)
print('Groq LLM loaded.')
```

```
Loading embedding model...
/usr/local/lib/python3.12/dist-packages/huggingface_hub/utils/_auth.py:93: UserWarning:
The secret `HF_TOKEN` does not exist in your Colab secrets.
To authenticate with the Hugging Face Hub, create a token in your settings tab (https://huggingface.co/settings/to
You will be able to reuse this secret in all of your notebooks.
Please note that authentication is recommended but still optional to access public models or datasets.
  warnings.warn(
Warning: You are sending unauthenticated requests to the HF Hub. Please set a HF_TOKEN to enable higher rate limit
WARNING:huggingface_hub.utils._http:Warning: You are sending unauthenticated requests to the HF Hub. Please set a
modules.json: 100%  349/349 [00:00<00:00, 20.6kB/s]
config_sentence_transformers.json: 100%  116/116 [00:00<00:00, 1.34kB/s]
README.md: 10.5k/? [00:00<00:00, 214kB/s]
sentence_bert_config.json: 100%  53.0/53.0 [00:00<00:00, 3.23kB/s]
config.json: 100%  612/612 [00:00<00:00, 50.4kB/s]
model.safetensors: 100%  90.9M/90.9M [00:01<00:00, 455MB/s]
Loading weights: 100%  103/103 [00:00<00:00, 384.94it/s, Materializing param=pooler.dense.weight]
BertModel LOAD REPORT from: sentence-transformers/all-MiniLM-L6-v2
Key                     | Status     |  |
------------------------+------------+--+-
embeddings.position_ids | UNEXPECTED |  |

Notes:
- UNEXPECTED    :can be ignored when loading from different task/architecture; not ok if you expect identical arch
tokenizer_config.json: 100%  350/350 [00:00<00:00, 11.8kB/s]
vocab.txt: 232k/? [00:00<00:00, 5.99MB/s]
tokenizer.json: 466k/? [00:00<00:00, 18.2MB/s]
special_tokens_map.json: 100%  112/112 [00:00<00:00, 10.2kB/s]
config.json: 100%  190/190 [00:00<00:00, 12.9kB/s]
Embedding model loaded.
Groq LLM loaded.
```

Step 3 — Multi File Type Loader

New in Week 3 — supports PDF, DOCX and TXT.

```
from langchain_community.document_loaders import PyMuPDFLoader
from langchain_community.document_loaders import Docx2txtLoader
from langchain_community.document_loaders import TextLoader
from langchain_text_splitters import RecursiveCharacterTextSplitter

def load_document(filepath):
    ext = os.path.splitext(filepath)[1].lower()

    if ext == '.pdf':
        loader = PyMuPDFLoader(filepath)
    elif ext == '.docx':
        loader = Docx2txtLoader(filepath)
    elif ext == '.txt':
        loader = TextLoader(filepath)
    else:
        raise ValueError(f'Unsupported file type: {ext}. Supported types: PDF, DOCX, TXT')
```

```
    docs = loader.load()

    if not docs:
        raise ValueError(f'Document is empty or could not be parsed: {filepath}')

    total_text = ''.join([d.page_content for d in docs]).strip()
    if len(total_text) < 100:
        raise ValueError(f'Document contains too little text to be useful: {filepath}')

    print(f'Loaded {len(docs)} pages from {os.path.basename(filepath)}')
    return docs

def chunk_documents(docs):
    # chunk_size=200 based on Week 2 finding
    splitter = RecursiveCharacterTextSplitter(
        chunk_size=200,
        chunk_overlap=50,
        length_function=len
    )
    chunks = splitter.split_documents(docs)
    print(f'Split into {len(chunks)} chunks')
    return chunks

print('Document loader ready.')
print('Supported file types: PDF, DOCX, TXT')
```

```
Document loader ready.
Supported file types: PDF, DOCX, TXT
```

Step 4 — Build FAISS Index

Updated from Week 1 — saves to per-document path under standalone/.

```
from langchain_community.vectorstores import FAISS
from tqdm import tqdm

def build_index(chunks, index_path):
    os.makedirs(index_path, exist_ok=True)

    batch_size = 50
    all_embeddings = []

    for i in tqdm(range(0, len(chunks), batch_size), desc='Embedding chunks', unit='batch'):
        batch = chunks[i:i + batch_size]
        batch_texts = [chunk.page_content for chunk in batch]
        batch_embeddings = embeddings.embed_documents(batch_texts)
        all_embeddings.extend(batch_embeddings)

    texts = [chunk.page_content for chunk in chunks]
    metadatas = [chunk.metadata for chunk in chunks]

    vectorstore = FAISS.from_embeddings(
        list(zip(texts, all_embeddings)),
        embeddings,
        metadatas=metadatas
    )

    vectorstore.save_local(index_path)
    print(f'Index saved: {index_path}')
    return vectorstore

print('Index builder ready.')
```

```
Index builder ready.
```

Step 5 — Upload and Index Multiple Documents

New in Week 3 — handles multiple files of different types in one upload.

```
from google.colab import files
import shutil

print('Upload one or more documents (PDF, DOCX, TXT)...')
uploaded = files.upload()

indexed_docs = {}

for filename in uploaded.keys():
    print(f'\nProcessing: {filename}')
    try:
        dest = f'{DOCS_DIR}/{filename}'
        doc_name = os.path.splitext(filename)[0]
        index_path = f'{STANDALONE_DIR}/{doc_name}'

        # Duplicate check
        if os.path.exists(index_path):
            response = input(f'"{doc_name}" is already indexed. Reindex? (y/n): ').strip().lower()
            if response != 'y':
                print(f'{doc_name} already indexed. Skipping.')
                continue

        shutil.copy(filename, dest)

        docs = load_document(dest)
        chunks = chunk_documents(docs)
        vectorstore = build_index(chunks, index_path)

        indexed_docs[doc_name] = vectorstore
        print(f'Successfully indexed: {doc_name} ({len(chunks)} chunks)')

    except ValueError as e:
        print(f'Skipped {filename}: {e}')

print(f'\nIndexing complete. {len(indexed_docs)} document(s) indexed:')
for name in indexed_docs:
    print(f'  - {name}')
```

```
Upload one or more documents (PDF, DOCX, TXT)...
Choose Files mars_lithograph  pdf.pdf
mars_lithograph pdf.pdf(application/pdf) - 335116 bytes, last modified: 4/13/2026 - 100% done
Saving mars_lithograph  pdf.pdf to mars_lithograph  pdf.pdf

Processing: mars_lithograph  pdf.pdf
"mars_lithograph  pdf" is already indexed. Reindex? (y/n): n
mars_lithograph  pdf already indexed. Skipping.

Indexing complete. 0 document(s) indexed:
```

Step 6 — List Available Indexes

New in Week 3 — shows all indexed documents.

```
def list_indexes():
    indexes = []
    if os.path.exists(STANDALONE_DIR):
        for name in os.listdir(STANDALONE_DIR):
            if os.path.isdir(f'{STANDALONE_DIR}/{name}'):
                indexes.append(name)

    if not indexes:
        print('No indexes found. Run Step 5 to upload and index documents.')
    else:
        print(f'Available documents ({len(indexes)}):')
```

```
        for i, name in enumerate(indexes):
            print(f'  [{i}] {name}')

    return indexes

available = list_indexes()
```

```
Available documents (3):
  [0] earth_lithograph
  [1] mars_lithograph  pdf
  [2] mars_lithograph
```

Step 6b — Delete Index

```
def delete_index(doc_name):
    index_path = f'{STANDALONE_DIR}/{doc_name}'

    if not os.path.exists(index_path):
        print(f'Index not found: {doc_name}')
        return

    response = input(f'Are you sure you want to delete "{doc_name}"? (y/n): ').strip().lower()
    if response != 'y':
        print('Deletion cancelled.')
        return

    import shutil
    shutil.rmtree(index_path)
    print(f'Deleted index: {doc_name}')

# Run list_indexes first to see available documents
available = list_indexes()

# Uncomment and set the name to delete
delete_index('mars_lithograph (1)')  # set document name to delete
```

```
Available documents (4):
  [0] earth_lithograph
  [1] mars_lithograph  pdf
  [2] mars_lithograph
  [3] mars_lithograph (1)
Are you sure you want to delete "mars_lithograph (1)"? (y/n): y
Deleted index: mars_lithograph (1)
```

Step 7 — Load a Single Index

New in Week 3 — select one document to query.

```
# Change this to the name from the list above
INDEX_NAME = 'mars_lithograph'  # set to the document name you want to query

index_path = f'{STANDALONE_DIR}/{INDEX_NAME}'
vectorstore = FAISS.load_local(
    index_path,
    embeddings,
    allow_dangerous_deserialization=True
)

print(f'Loaded index: {INDEX_NAME}')
```

```
Loaded index: mars_lithograph
```

Step 8 — Load Multiple Indexes and Merge

New in Week 3 — merge selected indexes for cross-document querying.

```
# Add the names of documents you want to query across
```

```
SELECTED_INDEXES = ['mars_lithograph','earth_lithograph']  # add document names e.g. ['doc1', 'doc2']  # add more

print(f'Merging {len(SELECTED_INDEXES)} indexes...')

merged_vectorstore = None

for name in SELECTED_INDEXES:
    index_path = f'{STANDALONE_DIR}/{name}'
    vs = FAISS.load_local(
        index_path,
        embeddings,
        allow_dangerous_deserialization=True
    )
    if merged_vectorstore is None:
        merged_vectorstore = vs
    else:
        merged_vectorstore.merge_from(vs)
    print(f'Merged: {name}')

print(f'\nMerged index ready. Querying across: {SELECTED_INDEXES}')
```

```
Merging 2 indexes...
Merged: mars_lithograph
Merged: earth_lithograph

Merged index ready. Querying across: ['mars_lithograph', 'earth_lithograph']
```

Step 9 — Build RAG Chain

Reused from Week 2 — no changes.

```
from langchain_core.prompts import PromptTemplate
from langchain_core.runnables import RunnablePassthrough, RunnableLambda
from langchain_core.output_parsers import StrOutputParser

template = """
You are a helpful assistant that answers questions based strictly on the provided context.
If the answer is not found in the context, say "I could not find an answer in the document."
Do not use any knowledge outside of the provided context.

Context:
{context}

Question:
{question}

Answer:
"""

prompt = PromptTemplate(
    input_variables=['context', 'question'],
    template=template
)

def format_chunks(docs):
    return '\n\n'.join([
        f'[Page {doc.metadata.get("page", 0) + 1}]\n{doc.page_content}'
        for doc in docs
    ])

def safe_ask(question, rag_chain):
    if not question.strip():
        print('Please enter a valid question.')
        return None
    answer = rag_chain.invoke(question)
    return answer

def build_rag_chain(vectorstore):
    retriever = vectorstore.as_retriever(search_kwargs={'k': 5})
```

```
    chain = (
        {
            'context': RunnableLambda(lambda q: retriever.invoke(q)) | format_chunks,
            'question': RunnablePassthrough()
        }
        | prompt
        | llm
        | StrOutputParser()
    )
    return chain, retriever

print('RAG chain builder ready.')
```

```
RAG chain builder ready.
```

Step 10 — Query Single Document

Updated from Week 2 — uses build_rag_chain function.

```
# Uses vectorstore from Step 7
rag_chain, retriever = build_rag_chain(vectorstore)

question = "Temperature of mars?"  # enter your question here

answer = safe_ask(question, rag_chain)

if answer is None:
    pass
else:
    source_docs = retriever.invoke(question)
    print(f'Question: {question}')
    print(f'Document: {INDEX_NAME}')
    print(f'\nAnswer: {answer}')
    print(f'\nSources:')
    for i, doc in enumerate(source_docs[:3]):
        source_file = doc.metadata.get('source', 'unknown')
        print(f'  [{i+1}] {os.path.basename(source_file)} | Page {doc.metadata.get("page", 0) + 1}: {doc.page_cor
```

```
Question: Temperature of mars?
Document: mars_lithograph

Answer: The temperature range of Mars is -87 to -5 deg C (-125 to 23 deg F).

Sources:
  [1] mars_lithograph.docx | Page 1: Temperature Range  -87 to -5 deg C (-125 to 23 deg F)

        Known Moons*    2

        Rings   0

*As of July 2013.


SIGNIFICANT DATES

...
  [2] mars_lithograph.docx | Page 1: Mars




www.nasa.gov...
  [3] mars_lithograph.docx | Page 1: The cold temperatures and thin atmosphere on Mars do not allow liquid water t
```

Step 11 — Query Across Multiple Documents

New in Week 3 — uses merged vectorstore from Step 8.

```
# Uses merged_vectorstore from Step 8
merged_chain, merged_retriever = build_rag_chain(merged_vectorstore)

question = "Is Earth bigger than Mars"  # enter your question here

answer = safe_ask(question, merged_chain)

if answer is None:
    pass
else:
    source_docs = merged_retriever.invoke(question)
    print(f'Question: {question}')
    print(f'Querying across: {SELECTED_INDEXES}')
    print(f'\nAnswer: {answer}')
    print(f'\nSources:')
    for i, doc in enumerate(source_docs[:3]):
        source_file = doc.metadata.get('source', 'unknown')
        print(f'  [{i+1}] {os.path.basename(source_file)} | Page {doc.metadata.get("page", 0) + 1}: {doc.page_cont
```

```
Question: Is Earth bigger than Mars
Querying across: ['mars_lithograph', 'earth_lithograph']

Answer: Yes, Earth is bigger than Mars. According to the context, Mars is described as "about half the size of Ear

Sources:
  [1] mars_lithograph.docx | Page 1: Mars is a rocky body about half the size of Earth. As with the other terrestr
  [2] earth_lithograph.pdf | Page 2: the environment, economy, and society.
Earth is the third planet from the Sun and the fifth largest in the
solar syste...
  [3] mars_lithograph.docx | Page 1: Mars




www.nasa.gov...
```